

Algorithms for Data Science

Lecture #10: Gradient Descent and Neural Network Training

Mateusz Buczynski

University of Warsaw

03.06.2026

Thanks to dr Krzysztof Fleszar for the base material.

Machine Learning as Optimization

Core idea

Training a model means choosing parameters that make prediction error small.

Algorithmic translation

We must minimize a function of many variables.

Ingredients of Supervised Learning

Data

Training examples

$$(x_1, y_1), \dots, (x_n, y_n).$$

Model

Produces prediction $\hat{y} = f_{\theta}(x)$.

Parameters

θ are the weights and biases we want to learn.

Loss Function

Purpose

Measure how bad the predictions are.

Examples

- squared error for regression
- cross-entropy for classification

Training objective

Minimize the loss over the training data.

Why Not Just Try All Parameters?

Problem

The parameter space is usually huge and continuous.

Example

A neural network can have millions or billions of parameters.

So

We need an iterative optimization algorithm, not brute force.

Gradient: Local Slope Information

For one variable

The derivative tells us the local slope of the loss.

For many variables

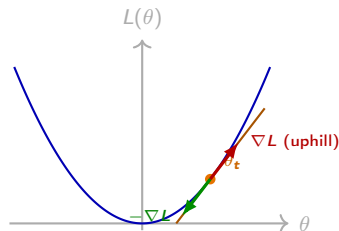
The gradient

$$\nabla L(\theta)$$

collects all partial derivatives.

Meaning

It points in the direction of steepest increase.



Why We Move in the Negative Gradient Direction

If the gradient points uphill

Then

$$-\nabla L(\theta)$$

points downhill.

Intuition

Take a small step in the direction that decreases the loss fastest locally.

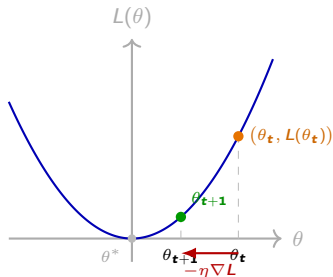
Gradient Descent Update Rule

Basic algorithm

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Parameters

- θ_t : current parameters
- η : learning rate (step size)
- $\nabla L(\theta_t)$: current gradient



Worked Example: Two Gradient Steps

Setup

$$L(w) = (w - 3)^2, \quad \nabla L(w) = 2(w - 3), \quad \eta = 0.5, \quad w_0 = 0$$

Step 1

$$\begin{aligned}\nabla L(w_0) &= 2(0 - 3) = -6 \\ w_1 &= 0 - 0.5 \cdot (-6) = 3\end{aligned}$$

Step 2

$$\begin{aligned}\nabla L(w_1) &= 2(3 - 3) = 0 \\ w_2 &= 3 - 0.5 \cdot 0 = 3\end{aligned}$$

Learning Rate

Too small

Very slow progress.

Too large

Overshooting, oscillation, or divergence.

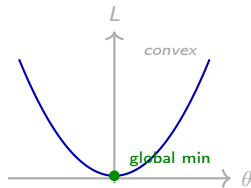
Engineering reality

Learning rate is one of the most important hyperparameters.

Convex vs Non-Convex Optimization

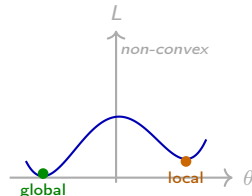
Convex loss

No bad local minima; gradient descent finds the global optimum.



Neural networks

Usually non-convex; may converge to a local minimum.



Example: Linear Regression

Model

$$\hat{y} = wx + b$$

Loss

Typically mean squared error.

Training

Gradient descent adjusts w and b to fit the data.

One Neuron as a Building Block

Computation

$$z = w^T x + b, \quad a = \sigma(z)$$

Interpretation

Weights combine inputs, bias shifts them, activation introduces nonlinearity.

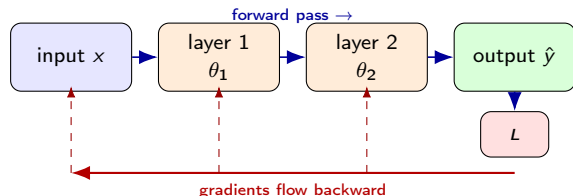
Why Neural Networks Need Backpropagation

Problem

The loss depends on parameters through many layers of intermediate computations.

Need

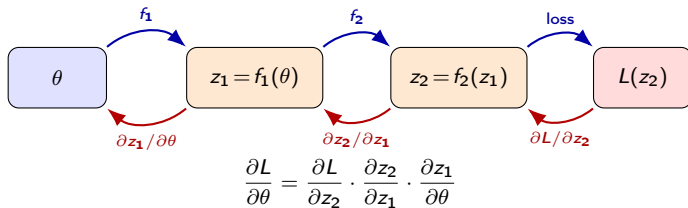
We must compute gradients of the final loss with respect to every parameter efficiently.



Chain Rule: The Core Mathematical Tool

Main principle

If a quantity depends on another quantity, derivatives multiply through the computational chain.



In neural networks

Backpropagation is repeated chain-rule application from the output back to earlier layers.

Backpropagation Intuition

Forward pass

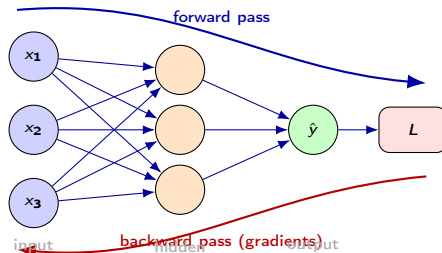
Compute predictions and the loss.

Backward pass

Propagate gradient information backward to compute parameter updates.

Why it matters

Without backpropagation, training deep networks would be far too expensive.



Backprop Walkthrough: Network Setup

Minimal network: one hidden unit



Formulas

$$z_1 = w_1 x \quad h = \sigma(z_1)$$

$$\hat{y} = w_2 h \quad L = \frac{1}{2}(\hat{y} - y)^2$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Concrete values

$$x = 1, \quad y = 0, \quad \eta = 0.5$$

$$w_1 = 1, \quad w_2 = 2$$

Backprop Step 1 — Forward Pass

Compute and store every intermediate value

$$z_1 = w_1 \cdot x = 1 \cdot 1 = 1$$

$$h = \sigma(z_1) = \sigma(1) \approx 0.73$$

$$\hat{y} = w_2 \cdot h = 2 \cdot 0.73 = 1.46$$

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(1.46)^2 \approx 1.07$$

Sigmoid at $z_1 = 1$

$$\sigma'(1) = 0.73 \cdot (1 - 0.73) \approx 0.20$$

Why store intermediates?

The backward pass reuses h , $\hat{y}$, and $\sigma'(z_1)$ — don't recompute.

Backprop Step 2 — Output Layer Gradient

Gradient of L with respect to w_2

$$\frac{\partial L}{\partial w_2} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\hat{y} - y} \cdot \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_h$$

Numbers

$$\frac{\partial L}{\partial w_2} = (1.46 - 0) \cdot 0.73 \approx 1.07$$

Save the upstream error signal

Define $\delta = \frac{\partial L}{\partial \hat{y}} = 1.46$. This scalar is passed backward into the hidden layer.

Backprop Step 3 — Hidden Layer Gradient

Chain rule extends back through h and z_1

$$\frac{\partial L}{\partial w_1} = \underbrace{\delta}_{1.46} \cdot \underbrace{\frac{\partial \hat{y}}{\partial h}}_{w_2=2} \cdot \underbrace{\frac{\partial h}{\partial z_1}}_{\sigma'(1) \approx 0.20} \cdot \underbrace{\frac{\partial z_1}{\partial w_1}}_{x=1}$$

Numbers

$$\frac{\partial L}{\partial w_1} = 1.46 \cdot 2 \cdot 0.20 \cdot 1 \approx 0.58$$

Pattern for deeper networks

Each extra layer adds one more local derivative to the product. The same δ is reused — this is why one backward pass suffices.

Backprop Step 4 — Weight Update

Apply gradient descent ($\eta = 0.5$)

$$w'_2 = w_2 - \eta \frac{\partial L}{\partial w_2} = 2 - 0.5 \cdot 1.07 = 1.47$$

$$w'_1 = w_1 - \eta \frac{\partial L}{\partial w_1} = 1 - 0.5 \cdot 0.58 = 0.71$$

Sanity check

Both weights decreased, so the network will predict a smaller $\hat{y}$, moving closer to the target $y = 0$.

One training iteration = four steps

Forward pass $\rightarrow$ loss $\rightarrow$ backward pass $\rightarrow$ update. Then repeat.

Training Loop for a Neural Network

- 1 initialize parameters
- 2 do a forward pass
- 3 compute the loss
- 4 do backpropagation
- 5 update parameters with gradient descent
- 6 repeat many times

Batch Gradient Descent

Rule

Use the full dataset to compute one exact gradient step.

Pros and cons

Stable but expensive for large datasets.

Stochastic Gradient Descent (SGD)

Rule

Use one randomly chosen example for each update.

Effect

Cheaper updates, but noisy gradient estimates.

Practical benefit

Can start improving quickly even on massive datasets.

Mini-Batch Gradient Descent

Compromise

Use a small batch of examples for each update.

Why this dominates practice

It balances hardware efficiency, gradient quality, and memory usage.

Common Training Problems

- loss decreases too slowly
- training becomes unstable
- gradients explode or vanish
- model overfits the training set

Message

Getting gradient descent to work well is both mathematics and engineering.

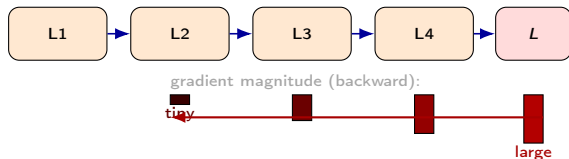
Vanishing and Exploding Gradients

What can go wrong

As gradients are propagated through many layers, they can shrink toward zero or grow uncontrollably.

Consequence

Early layers may learn too slowly or training may become numerically unstable.



Regularization and Generalization

Training loss is not the whole story

A model can fit the training data well but still perform poorly on unseen data.

Typical tools

Weight decay, dropout, early stopping, data augmentation.

When Do We Stop Training?

Typical criteria

- fixed number of epochs
- validation loss stops improving
- gradients or updates become very small

Best practice

Watch validation performance, not only training loss.

What to Remember

- Gradient descent is the standard template for training differentiable models.
- The gradient gives local information for improving the parameters.
- Backpropagation computes gradients efficiently in layered models.
- Mini-batch SGD is the practical default for large-scale neural training.
- Good training requires both optimization insight and engineering care.